

Chapter 01: Python: What It Is and How It Speaks

What Python is • Who made it • Why we use it • Which version • print, variables, operators, `type()`, errors

Chapter 01 of 10

Dr. Brahim Zirari

Yahia Farès University of Medea

[in linkedin.com/in/brahimzirari](https://in.linkedin.com/in/brahimzirari) · [Google Scholar](#)

October 2026

1. What is Python?

❑ **Definition** – Python is a **programming language**: a set of words and rules you use to tell a computer what to do.

❑ **Its three labels** – You will see Python described with these three words. They are worth understanding now, because they explain how it behaves.

- **High-level** — the code is close to English and far from the machine. You write `price * 1.19`, not instructions about registers and memory addresses.
- **Interpreted** — a program called the *interpreter* reads your file and runs it line by line. There is no separate “build” step to wait for, and a mistake on line 40 is only discovered when line 40 runs.
- **General-purpose** — it is not tied to one subject. The same language serves a website, a risk model and a game.

❑ **Who made it** – **Guido van Rossum**, a Dutch programmer, began Python in **December 1989** as a hobby project during the Christmas holiday. The first public version, 0.9.0, appeared in **1991**.

Remark: the name comes from the British comedy show Monty Python’s Flying Circus, which Guido was watching at the time. It is not named after the snake.

❑ **Who owns it today** – Nobody owns Python. It belongs to the **Python Software Foundation (PSF)**, a non-profit. Guido led the language for almost 30 years with the title *BDFL* — Benevolent Dictator For Life — and stepped down in 2018. A small elected **Steering Council** now decides what goes into the language.

2. Why we use Python

- **Free and open source.** You pay nothing, on any operating system, and you may read the source code of the language itself.
- **The most used language in the world.** In the TIOBE index of September 2026 Python is first with **17.76%**, ahead of C, C++, Java and C#.
- **A very large community.** Almost every error message you will meet has already been asked and answered online.

- **Short and readable.** The same task needs fewer lines than in Java or C++.
- **It runs everywhere.** Windows, macOS, Linux — the same file.
- **Ready-made libraries.** Thousands of free libraries: NumPy and pandas for data, Matplotlib for charts, scikit-learn for machine learning.

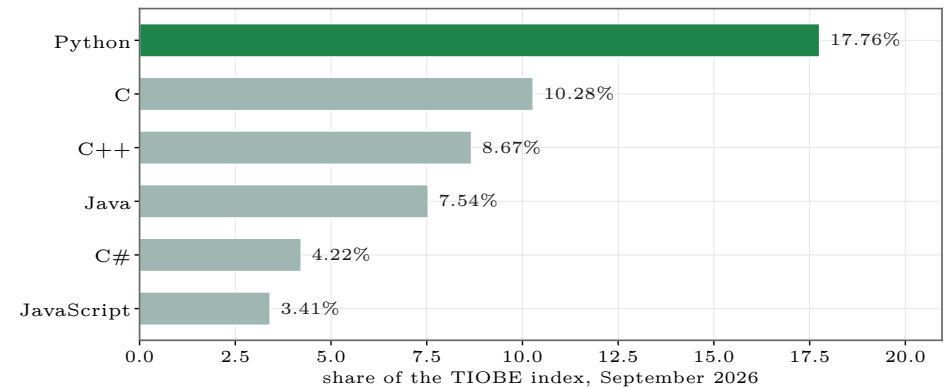


Figure 2. Python’s lead is not small: it is larger than C and C++ together.

❑ **Why it matters for FinTech** – Finance work is almost always the same five steps, and Python does all five in one language. It is the standard tool for risk calculation, pricing, fraud detection, credit scoring, algorithmic trading and automatic reporting.

3. Python compared with other languages

❑ **The comparison** – Every language trades one advantage for another. Python’s trade is clear: it gives up raw speed to gain short, readable code.

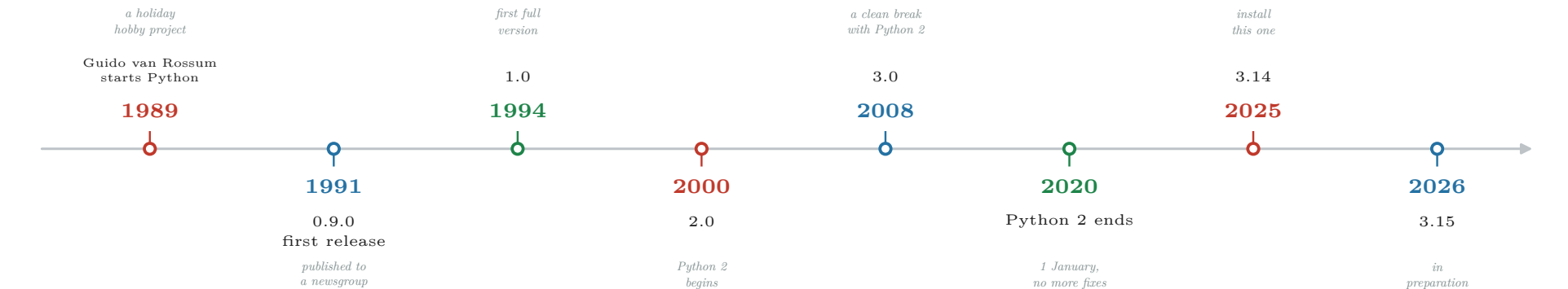


Figure 1. The history of Python in eight dates.

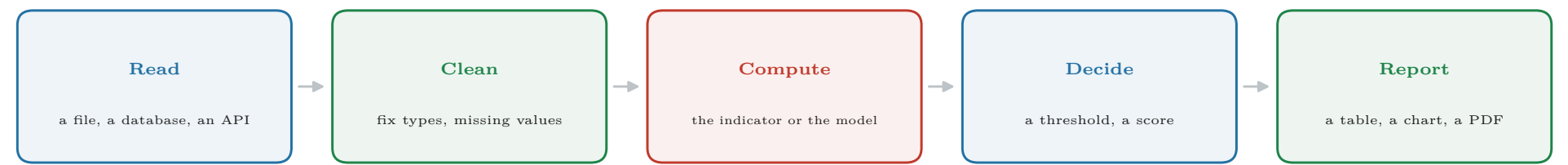


Figure 3. The five steps of almost every piece of finance data work. One language covers all of them.

Language	Its advantage over Python	Python’s strength
Java	Faster execution, static typing	Shorter code, dynamic typing, good glue language
Perl	Powerful text processing, regex built in	More readable, better structure, wider use
Tcl	Simple, string-based	Richer data types, modular
Smalltalk	Everything is an object, refined collections	Familiar syntax, file-based modules, strong internet libraries
C++	High performance, low-level control	Much shorter code; excellent as a glue language

❑ **Glue language** – A **glue language** is one that is easy to use to connect, coordinate and control programs written in other languages. Python is used this way constantly: the heavy calculation runs in C, and Python gives the orders.

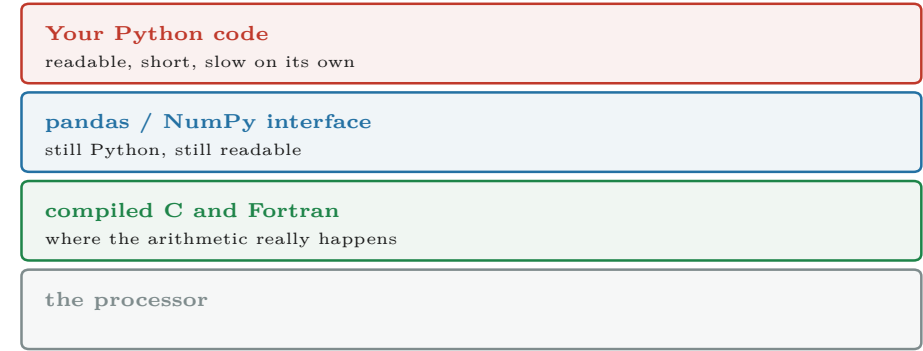


Figure 4. Why “Python is slow” is only half true: you write the top layer, the machine runs the third one.

4. Where Python is used

❑ **Not one field** – Python is used across eight broad areas, listed in Figure 5. Three of them concern you directly: **data science**, **machine learning** and **automation**. The library names shown there are the ones you will meet first in each field — you are not expected to know any of them yet.

↔ official list: www.python.org/about/apps/

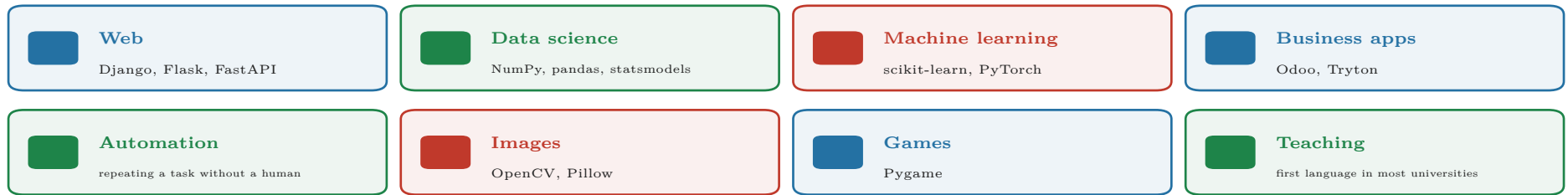


Figure 5. Eight fields, one language.

5. Python 2 or Python 3?

❑ **Use Python 3** – Python 3 is **not backward compatible** with Python 2: a Python 2 file does not necessarily run under Python 3. Python 2 stopped being supported on **1 January 2020** and must not be used for new work.

Pitfall: old tutorials still show `print "hello"` without brackets. That is Python 2 and it is a syntax error in Python 3. The correct form is `print("hello")`. If a tutorial does this, it is out of date — find another one.

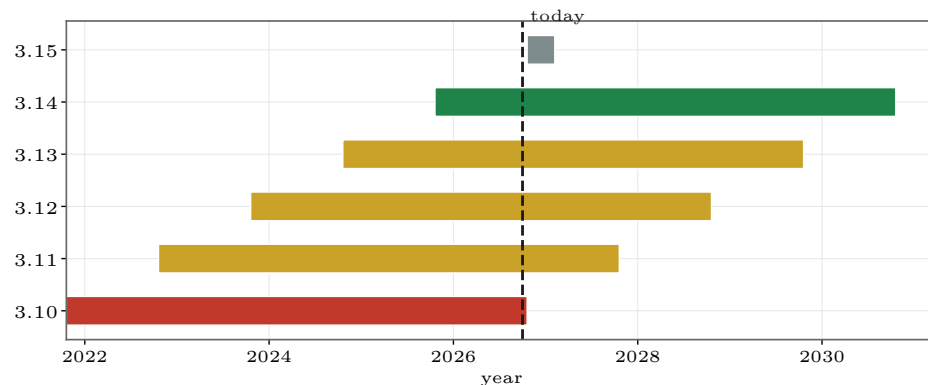


Figure 6. Support windows. Green still receives bug fixes, yellow only security fixes, red receives nothing at all.

❑ **Which version to install** – **Python 3.14**. It is the current version and it still receives bug fixes. 3.13 is acceptable for an existing project; anything 3.10 or older receives nothing, and 2.7 has been dead since 2020.

↪ downloads: www.python.org/downloads/

6. Your first program

❑ **print()** – **print** is a **function**: a named piece of work you ask Python to do. You call it by writing its name followed by **round brackets**, with what you want inside them.

```
print("Hello, FinTech")
print(2026)
print(1250 * 0.19)
```

Output

```
Hello, FinTech
2026
237.5
```

❑ **Three things to notice** – Text must be in **quotes**; numbers must not. `print()` finishes with a new line, so each call puts its result on its own line. And a calculation inside the brackets is done *first*, then printed.

❑ **Printing several things at once** – Separate them with commas. Python inserts one space between them.

```
print("Interest:", 237.5, "DZD")
```

Output

```
Interest: 237.5 DZD
```

❑ **Comments** – A **#** makes Python ignore the rest of the line. Comments are for the human reading the file — usually you, three weeks later.

```
# VAT rate applied to banking fees in 2026
print(1250 * 0.19) # 19% of the fee
```

Pitfall: a comment is not a translation of the code. # add one to x next to `x = x + 1` says nothing. # accounts start at 1, not 0 says something.

7. Variables

❑ **Definition** – A **variable** is a name that refers to a value. You create one with the **=** sign: the value on the right is computed first, then the name on the left is made to refer to it.

```
capital = 250000
rate = 0.035
interest = capital * rate

print("Interest:", interest)
print("Total:", capital + interest)
```

Output

```
Interest: 8750.0
Total: 258750.0
```

Pitfall: = is not equality. capital = 250000 is an order: “make the name capital refer to 250000”. It is read right-to-left. Testing whether two things are equal uses ==, which comes in Chapter 08.

❑ **Why bother** – Three reasons, in order of importance. A name says what a number means, so `capital * rate` is readable where `250000 * 0.035` is not. A value written once can be changed once. And a result you have computed can be used again without computing it twice.

❑ **Naming rules** – These are enforced by Python, not by style:

Allowed	letters, digits and <code>_</code> ; must not <i>start</i> with a digit
Case matters	<code>Rate</code> , <code>rate</code> and <code>RATE</code> are three different variables
Not allowed	the language’s own words: <code>print</code> is overwritable but <code>if</code> , <code>for</code> , <code>class</code> , <code>return</code> , <code>None</code> are not
Convention	lowercase with underscores: <code>interest_rate</code> , <code>total_due</code>

❑ **f-strings** – Putting an `f` before the opening quote lets you place a variable, or a whole calculation, inside the text in curly brackets. This is the form you should use from now on.

```
capital = 250000
rate = 0.035
print(f"{capital} DZD at {rate} gives {capital * rate} DZD")
print(f"rounded: {capital * rate:.2f} DZD")
```

Output

```
250000 DZD at 0.035 gives 8750.0 DZD
rounded: 8750.00 DZD
```

Remark: the `:.2f` after the expression means “show it as a decimal number with 2 digits after the point”. For money, always do this.

8. Arithmetic operators

❑ **The seven** – Python has seven arithmetic operators. Three of them do division, and the difference between those three is examined in every exam.

Name	What it does	Example
<code>+</code>	addition	$7 + 2 \rightarrow 9$
<code>-</code>	subtraction	$7 - 2 \rightarrow 5$
<code>*</code>	multiplication	$7 * 2 \rightarrow 14$
<code>/</code>	true division	$7 / 2 \rightarrow 3.5$
<code>//</code>	floor division	always gives a decimal number throws the fraction away $7 // 2 \rightarrow 3$
<code>%</code>	modulo	gives what is left over $7 \% 2 \rightarrow 1$
<code>**</code>	power	$7 ** 2 \rightarrow 49$

❑ **The identity that ties them together** – For any whole numbers a and b with $b \neq 0$:

$$a = b \times (a // b) + (a \% b)$$

where a is the amount being divided, b the number of parts, $a // b$ the whole amount in each part, and $a \% b$ what cannot be divided. Check with $a = 7450$, $b = 4$: $4 \times 1862 + 2 = 7450$.

❑ **Order of operations** – The usual mathematical order applies: `**` first, then `*` / `//` `%`, then `+` `-`. Use round brackets when you want a different order — and use them even when you do not need them, if they make the line clearer.

```
print(250000 + 250000 * 0.035) # multiplies first
print((250000 + 250000) * 0.035) # brackets first
```

Output

```
258750.0
17500.0
```

Pitfall: `0.1 + 0.2` does not print `0.3`. It prints `0.30000000000000004`. Computers store decimals in binary and cannot represent `0.1` exactly. This is not a Python fault — it is true of every language. The cure is to `round()` before showing a number to a human, never to pretend it did not happen.

9. Types

❑ **type()** – Every value in Python has a **type**, and **type** tells you which. Four types matter in this chapter:

Type	Name	What it holds
<code>int</code>	integer	a whole number: 250000, -3
<code>float</code>	floating point	a decimal number: 0.035, 8750.0
<code>str</code>	string	text in quotes: "Amina", "250000"
<code>bool</code>	boolean	exactly True or False

```
print(type(250000))
print(type(0.035))
print(type("250000"))
print(type(7 / 1))
```

Output

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'float'>
```

Remark: the last line is the one to remember: `7 / 1` is `7.0`, a float, not an int. `/` always produces a float, even when the division is exact.

❑ **Converting** – `int`, `float` and `str` also work as converters. They do not change the original value; they produce a new one.

```
print(int("42") + 8)
print(float("3.5") * 2)
print("Total: " + str(1250))
```

Output

```
50
7.0
Total: 1250
```

Pitfall: "250000" with quotes is text, not a number. "25" + "3" gives "253", because + on two strings joins them. If a number arrives as text — and everything typed by a user does — convert it before you calculate with it.

10. Reading from the keyboard

❑ **input()** – `input` shows a message, waits for the user to type something and press Enter, and hands back **what they typed as a string**. Always.

```
capital = float(input("Capital in DZD: "))
rate = float(input("Annual rate, e.g. 0.035: "))
print(f"Interest: {capital * rate:.2f} DZD")
```

*Pitfall: without float() around input() this program is wrong, not broken: "250000" * 0.035 raises an error, but "250000" * 3 silently prints the text three times over. The quietest bugs are the dangerous ones.*

11. Reading an error message

❑ **Errors are information** – An error message is Python telling you precisely what it could not do and where. It is not a punishment, and it is not an interruption of the work — reading it *is* the work.

A traceback, line by line

```
Traceback (most recent call last):
  File "fees.py", line 3, in <module>
    total = capitl * rate
           ^^^^^
NameError: name 'capitl' is not defined
```

line 1 a fixed heading; ignore it
 line 2 *where*: the file `fees.py`, line 3
 line 3 the offending line, copied from your file
 line 4 carets under the exact part that failed
 line 5 the actual error — its kind, then its detail

❑ **Read it from the bottom up** – The last line is the error. Here it is a `NameError`: `capitl` was never created — the name `capital` was typed wrong. The three kinds you will meet this week:

<code>SyntaxError</code>	the line is not Python at all: a missing bracket or quote, or <code>print "x"</code> in Python 2 style. Nothing runs.
<code>NameError</code>	a name is used before it exists: a typo, or a variable created after the line that uses it.
<code>TypeError</code>	the operation does not fit the types: <code>"25" * 0.035</code> , text multiplied by a decimal.

12. How to learn programming

❑ **Two halves, both required** – Learning to code is not the same as practising. You need both.

Learn coding	1. Write code yourself. 2. Read and understand code written by others. 3. Fix errors.
Practise	Repeat on new problems until the syntax is automatic.

Pitfall: Do not copy and paste code. Typing it yourself is what builds the memory. Copied code also hides errors: you cannot fix what you did not write.

❑ **A note for data work** – A data analyst is expected to know at least four tools: a spreadsheet (Excel), a query language (SQL), a programming language (Python), and a reporting tool. Python is the one that connects the other three.

What you can do after this chapter

- Say what Python is, who made it, who runs it now, and which version to install.
- Print text, numbers and the result of a calculation.
- Create variables, name them legally, and use them in an f-string.

- Use all seven arithmetic operators, and explain the difference between `/`, `//` and `%`.
- Name the type of a value with `type()`, and convert between `int`, `float` and `str`.
- Read a value from the keyboard and convert it before calculating.
- Read a traceback and name the three commonest errors.

Quick reference — everything in this chapter

Output and comments

<code>print(x)</code>	show <code>x</code> , then move to a new line
<code>print(a, b)</code>	show both, separated by one space
<code># text</code>	Python ignores the rest of the line

Variables

<code>name = value</code>	make <code>name</code> refer to <code>value</code> ; read right-to-left
<code>f"...{x}..."</code>	put the value of <code>x</code> inside the text
<code>f"{x:.2f}"</code>	... with exactly two decimals. Use for money

Arithmetic

<code>+</code>	add
<code>-</code>	subtract
<code>*</code>	multiply
<code>**</code>	power
<code>/</code>	division; <i>always</i> gives a float
<code>//</code>	whole part of the division
<code>%</code>	remainder of the division

Types

<code>type(x)</code>	name the type of <code>x</code>
<code>int</code>	whole number, decimal number
<code>float</code>	text in quotes, <code>True/False</code>
<code>str</code>	text in quotes, <code>True/False</code>
<code>bool</code>	text in quotes, <code>True/False</code>
<code>int(x)</code>	convert; the original is unchanged
<code>float(x)</code>	convert; the original is unchanged
<code>str(x)</code>	convert; the original is unchanged

Input

<code>input("msg")</code>	ask, wait, return what was typed as text
<code>float(input(...))</code>	the form you almost always want

The three first errors

<code>SyntaxError</code>	the line is not Python; nothing runs
<code>NameError</code>	the name does not exist; usually a typo
<code>TypeError</code>	the operation does not fit the types

❑ **Next** – Chapter 02 — where you actually type all of this: the interpreter, scripts, notebooks and the IDEs.

→ official tutorial: docs.python.org/3/tutorial/index.html

→ the language reference: docs.python.org/3/reference/index.html

→ exercises: www.geeksforgeeks.org/.../